AeroSentry Interceptor Perception Pipeline

Daniel Luzzatto

May 15, 2026 Github Repo

Abstract

This report presents a prototype perception pipeline for detecting airborne fixed-wing UAVs from an interceptor drone's onboard camera. The system combines YOLOv11n detection with SAHI sliced inference for small-object detection, CSRT visual tracking, and Kalman filter state estimation. Four training iterations were evaluated; the final model uses two-class discrimination (none vs. uav) with hard-negative mining to achieve near-zero false positives on deployment footage, at the cost of 33% recall, a tradeoff prioritizing precision for autonomous engagement safety. Key limitations include a domain gap between training data and deployment footage, and sub-real-time throughput (1.9 fps on CPU) pending Jetson Orin Nano TensorRT deployment.

1 PipelineOverview

The system architecture follows a multi-stage approach designed for high-speed aerial target tracking:

1. Ingestion and Pre-processing: Frames are normalized and enhanced to maintain consistency with pre-trained YOLOv11n weights and to improve feature contrast.
2. Detection and Tracking: The pipeline integrates deep-learning detection (YOLOv11n), visual tracking (CSRT), and state estimation (Kalman Filter). The Kalman Filter predicts the target's future state and dynamically defines a Region of Interest (ROI) to accelerate detection. YOLOv11n was selected for its 2.6M parameter efficiency, balancing the accuracy required for small targets with the low-latency demands of the Orin Nano. Its ecosystem provides reliable TensorRT export and quantization paths, which are critical for edge deployment. To maintain a high frame rate, I traded off model complexity for a hybrid approach, using CSRT and Kalman filtering to bridge detection gaps caused by motion blur or temporary occlusions.
3. Verification: Bounding boxes undergo validation via absolute size, size variation, absolute position, while the kalman prediction is also checked for motion, ensuring the prediction is not too far from the detection.

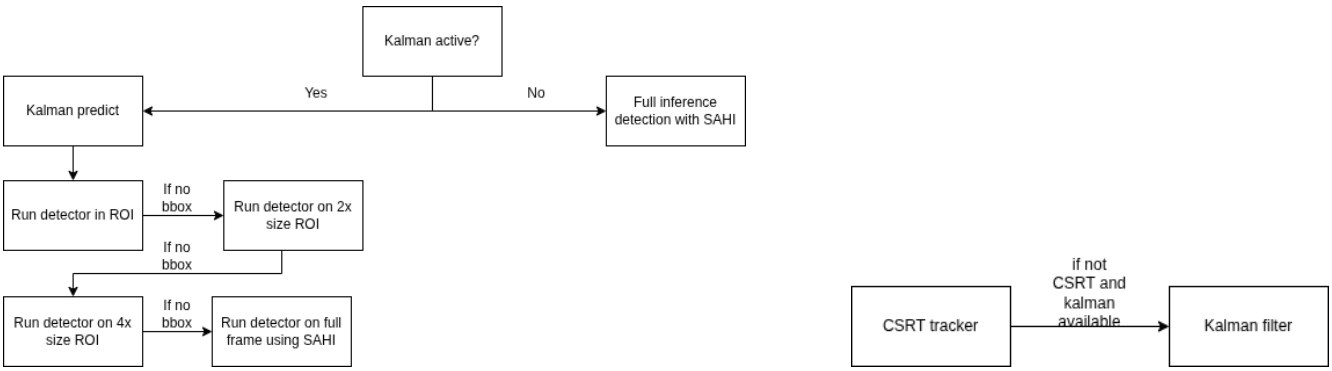
1.1 Pre-processing & Ingestion

To normalize sensor data for the model, a three-stage enhancement pipeline is implemented:

1. Channel Normalization: Single-channel inputs are replicated across three RGB planes to ensure compatibility with pre-trained weights and dataset images.
2. CLAHE: Contrast Limited Adaptive Histogram Equalization improves local contrast and enhances target edge definition against complex backgrounds.
3. Noise Reduction: Frames are convolved with a Gaussian kernel to suppress high-frequency noise and minimize false-positive artifacts.

1.2 Hybrid Tracking Strategy

The pipeline optimizes throughput by alternating between Inference and Non-Inference frames. YOLOv11n is utilized during inference cycles, while intermediate frames rely on a low-overhead combination of visual tracking and predictive state estimation.



(a) Inference Frame Search Strategy

(b) Non-Inference State Maintenance

Figure 1: Perception and Tracking Logic: (a) Hierarchical ROI cascading (1x to 4x) and full-frame fallback validated via size and motion gates. The inference frame frequency is reduced in case the detections have low score. (b) State maintenance using CSRT tracking with Kalman fallback. Tracks expire after set inactivity, triggering a velocity reset and a transition back to full-frame detection. The bounding box is saved as `Detection(frame_id, bbox, score, class, source)`, where class is always `airborne_fixed_wing` per the assignment specification.

1.3 Slicing Aided Hyper Inference (SAHI)

While the YOLOv11n model was trained on 640px inputs, the target video streams are significantly larger (e.g., 1280x720). To maintain high recall for small, distant objects without losing resolution, I utilize SAHI. This technique partitions the frame into smaller, partially overlapping tiles for localized inference. While SAHI improves small-object detection, it increases latency; as a consequence, the pipeline prioritizes the Kalman-derived ROI and only falls back to SAHI when a track is lost.

2 False Positive Reduction Logic

2.1 Physically-Informed Suppression

To minimize False Positives (FP), the system implements several physically-informed heuristic filters at inference:

1. Relative Size Gate: Detections are compared against a running weighted average of the target's area derived from the Kalman filter state. If a new detection's area deviates significantly from this expected mean, it is suppressed as a likely artifact. This saved about 7.8% of the positive detections in the demo video.

2. Morphological Constraints: Absolute size thresholds are applied to discard outliers (extremely large or small boxes). Additionally, the aspect ratio ($width/height$) is monitored; detections that are too vertical are rejected, as fixed-wing geometries typically yield a more horizontal bounding box.

3. Spatial Gating: Detections within the lower 35% of the frame are discarded. This region typically contains ground clutter, parked aircraft, or terrestrial noise that is irrelevant to the UAV-to-UAV tracking task. Note that this is a tunable parameter and it would likely need to become adaptable at production.

4. Adaptive Motion Gate: Kalman predictions that drift beyond a specific distance from the last known valid detection are rejected. The motion threshold is dynamically scaled based on the target's apparent size; this allows closer, larger targets a higher degree of pixel-travel while keeping tighter constraints on distant, smaller objects. This saved about 7.4% of the total Kalman predictions (see Figure 1b).

2.2 Adversarial Testing (Hard Negatives)

In order to quantify the model's ability to detect hard negatives, 132 airplanes images were mined from COCO (using the option shuffle therefore the expected number of overlapping images is 12.65). The model I trained (Model 4 from Table 1) shows 19 detections, making the FPs about 14.4% of the adversarial set size. This result shows that the model's knowledge and, paired with the FP suppression described in section 1.1, most of the FPs are avoided in the demo video. A fully rigorous independent evaluation would require a labeled video dataset with ground-truth FP annotations, which is left as a next step.

3 Evaluation and Diagnostics

3.1 Class Imbalance and Hard Negative Mining

The initial dataset showed a severe class imbalance, with nearly all training and validation samples containing positive targets (e.g., 2,835 positives out of 2,892 train samples), making the dataset insufficient for learning hard negative rejection across all splits. This leads to models biased toward positive predictions. To address this, I mined Hard Negatives by extracting images of birds, airplanes, and sky patches from the COCO dataset. The final dataset size of approximately 3,406 training samples was chosen to balance the capacity of the YOLOv11n architecture, preventing underfitting while providing enough diversity to avoid overfitting.

3.2 Model Performance Comparison & Results

Four distinct iterations were trained to evaluate the impact of data augmentation and pre-processing. The results are summarized in Table 1.

Table 1: Performance Comparison Across Model Iterations on the Images Test Set. (1) Baseline performance on provided data. (2) Integration of initial hard-negatives to reduce sky-artifact triggers. (3) Channel fix for hard negatives. (4) Final model utilizing refined class mapping (None vs. UAV) and bird hard negatives.
(1) Models 1 and 2 are included as baselines only; both failed to return any true positives.

Model	Samples	Negs.	Pre-Proc.	Epochs	mAP@50	mAP@50-95	Prec.	Rec.	F1
1. Baseline(1)	2,892	57	None	150	–	–	0%	0%	-
2. Pre-Proc.(1)	3,135	300	CLAHE, Blur	150%	–	–	0%	0%	-
3. Pre-Proc 2	3,135	300	CLAHE, Blur	150	93.6%	51.5%	91.3 %	86.6%	88.9%
4. Final	3,406	583	CLAHE, Blur	180	90.9%	50.5%	91.8%	82.3%	86.7%

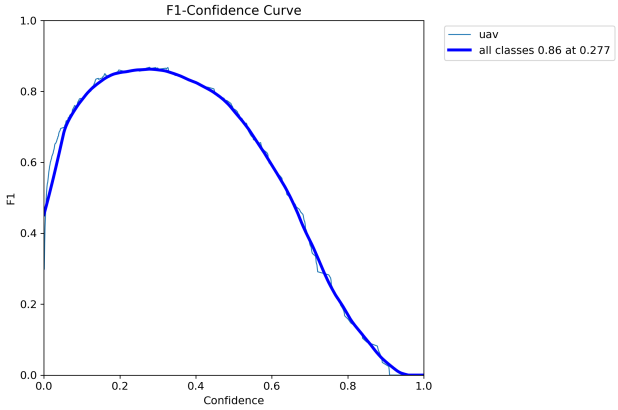
While Model 3 exhibits superior metrics on the static test set, Model 4 is selected for final deployment. Both iterations were benchmarked on the evaluation footage: Model 3 achieved a 64.1% raw detection rate, whereas Model 4 yielded 33.4%. However, qualitative frame-by-frame inspection revealed that about half of Model 3's detections were FPs triggered by sky artifacts.

In contrast, Model 4 demonstrated near-zero FPs, providing a significantly more reliable track. This divergence highlights a critical domain gap between the dataset and the footage: reducing it by using actual footage in training would improve the prediction significantly, particularly increasing the TPs.

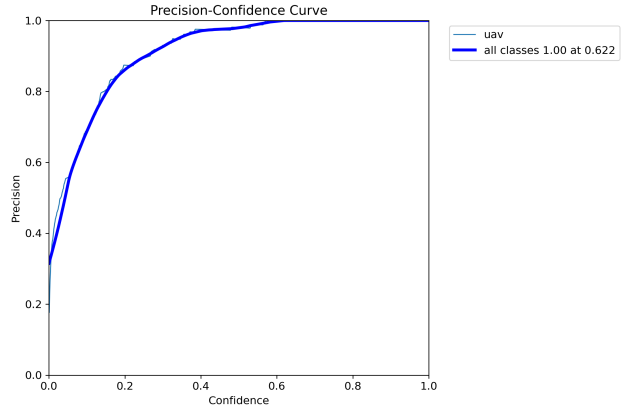
In an autonomous interceptor mission, Precision is the primary safety constraint. An FP would erroneously alter the interceptor's flight path, leading to a failed engagement or loss of kinetic energy. Given that the demo video always contains a drone, that Model 3 detects 64% of the frame (about half TPs and half FPs), and Model 4 detects 33.4% of the frames (almost all TPs), I can infer the following (qualitative) results:

Table 2: Qualitative Precision and Recall comparison derived from evaluation footage benchmarking.

Model	Precision	Recall	F1-Score
3. Augmented	$\approx$ 50%	$\approx$ 33%	$\approx$ 40%
4. Final	$\approx$ 100%	$\approx$ 33%	$\approx$ 50%



(a) F1-Confidence Curve: Optimal operational balance at 0.86 (conf $\approx$ 0.3).



(b) Precision-Confidence Curve: Reaches 1.00 Precision at a 0.622 threshold.

Figure 2: Analysis performed on the static test set images.

The performance of the final model on the test set illustrates a strategic trade-off between sensitivity and reliability. The F1-Confidence Curve (Figure 2a) identifies a peak of 0.86 at about 0.3 confidence threshold, which I utilize as the baseline for initializing temporal tracks.

To minimize FPs, the Precision-Confidence Curve (Figure 2b) is presented. The model achieves 100% precision at thresholds exceeding 0.622, though this will need to be inspected further on footage.

While the demo video is taken as main reference, the model was also tested on the other provided videos. In these the drone (if there is any) is very far and the pipeline struggles finding it. Here is a breakdown of the results, video by video:

Table 3: Qualitative results for the videos. (2) The assumption is that every frame shows a drone. However, I don't see it myself quite often, but it is difficult to quantify in how many frames it is there. (3) The video ran at 1.9 fps using the ONNX weights; 1.5 fps is the result using pt weights and it is reported for fair comparison with the other videos, which were tested with pt weights only.

Video	**Frame size**	**Frames / s**	**Detection %**	**Precision**	**Recall**	**F1-Score**
Demo Video	1280×720	1.5(3)	$\approx$ 33%	$\approx$ 100%	$\approx$ 33%	$\approx$ 50%
Clip 49	1920×1080	1.3	0%	0%	0%	–
Clip 50	1920×1080	0.6	$\approx$ 5.4%	$\approx$ 61.1%	$\approx$ 3.7% (2)	$\approx$ 7.2%

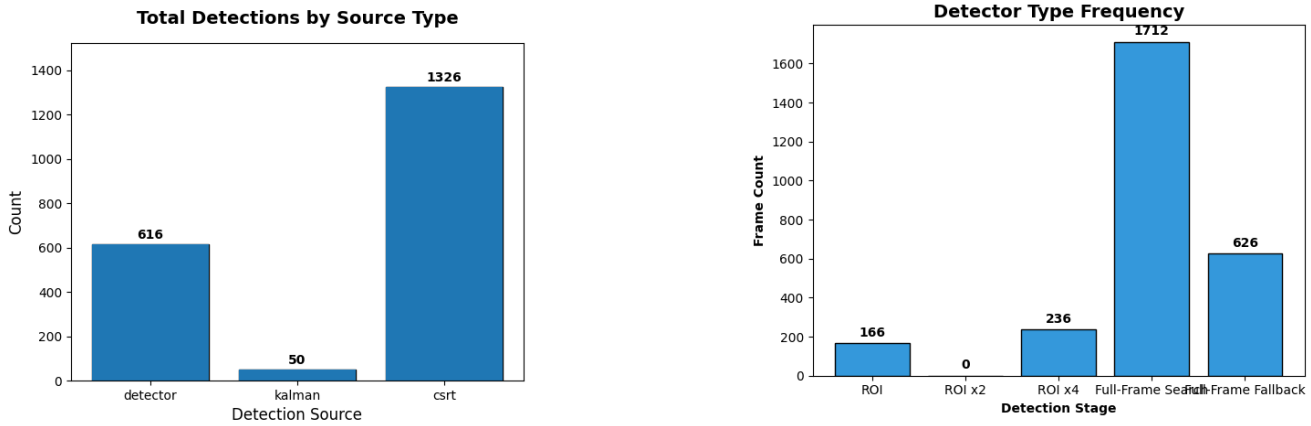
The significant performance drop in Clips 49 and 50 shows once more the necessity to train the model on actual footage. In these videos, the UAV occupies a substantially smaller pixel-area than in the 720p Demo Video, and the detector's feature extraction begins to degrade. While Clip 50 shows a 61.1% precision on the few successful detections, the low recall confirms that the model struggles finding the drone when it is very small and distant.

3.3 Qualitative Failure Analysis

- Demo video: There are a few reasons that cause failures in this video: often the UAV is in the lower part of the screen, or recorder from above, and the pipeline believes the drone is not airborne; Other times, the drone is far from the camera, and the detector struggles finding it; Moreover, sometimes the background throws off the drone and subtracting the average of the previous few (say 3 frames) could be helpful.

- Other videos: The low fps in these video is caused by the size, which requires more SAHI tiles; Regarding the performance, there are almost no detection, matching the videos (either there is no drone, or it is extremely far and I struggle to find it myself, there could be birds). In any case, making the SAHI tiles adaptive (the current pipeline hardcodes the size), as well as the motion, size gates could improve the performance.

3.4 Detections Analysis



(a) Positive detections type

(b) YOLO detections type.

Figure 3: (a) Bounding box types by frequency in the demo video. (b) YOLO detection type; Column 4 (full-frame-search) refers to the case in which there is no active Kalman filter (Kalman active "No" from Figure 1a), and 5 (frame-fallback) refers to the case in which all the ROI system did not work (last box in Figure 1a). The high YOLO usage is the performance bottleneck: need training data aligned to the footage, and model trained on full size picture (avoid SAHI). Moreover, ROI $\times 2$ was not useful, if this is confirmed for a larger video sample, it can be removed to decrease latency.

4 Edge Optimization: Jetson Orin Nano Deployment

4.1 Latency & Throughput Projections

The pipeline ran at 1.9 fps on the demo video (taken as reference). This is influenced by a number of factors:

1. As previously noted, SAHI inference is the primary speed bottleneck. The current configuration executes approximately 15–20 inference tiles per frame (256×256 pixels with 30% overlap). Fine-tuning the model on native 1280×720 pixel imagery would yield a 10–15$\times$ speedup by eliminating tile-based overhead. Furthermore, native training is expected to improve detection accuracy, allowing the pipeline to rely more extensively on the computationally efficient Kalman-tracker logic.

2. The current benchmarks were recorded using ONNX weights on an Intel i7-13700H CPU, which lacks the parallel throughput required for real-time YOLOv11 inference. In contrast, Jetson Orin Nano provides a dedicated compute budget of 40 TOPS (INT8) or 20 TFLOPS (FP16). By leveraging the Jetson's GPU, specifically optimized for matrix multiplications, a 3–5$\times$ speedup is anticipated for the inference and pre-processing stages. Furthermore, the Orin Nano utilizes a Unified Memory Architecture that allows further throughput increase when compared to traditional x86 PCIe overhead. Our pipeline will be optimized to utilize zero-copy memory pinned between the CPU and GPU to share the 8GB efficiently with the OS and high-speed frame buffers, eliminating latency derived from data transfer.

3. The baseline pipeline utilizes FP32 precision. Transitioning to FP16 quantization is expected to provide a 2$\times$ speedup with negligible impact on accuracy. Additionally, INT8 quantization could offer a 4$\times$ speedup, provided a calibration dataset is utilized to maintain detection sensitivity for small-scale targets.

4. While the CSRT tracker is highly precise, it is computationally expensive. Switching to a correlation-based filter like KCF or MOSSE would increase the frames-per-second during non-inference cycles. However, until the detection bottleneck is resolved, tracking speed remains a secondary concern.

The immediate priority is to implement the first three optimizations (Native Resolution, Hardware Acceleration, and FP16 Quantization). Once these measures are taken, the pipeline will approach real-time performance (30–60 FPS), though precise gains would require profiling on-device. Once these are deployed, a secondary profiling phase will be conducted to identify the new system bottleneck, specifically monitoring the frequency of the various detection methods to prioritize further algorithmic refinements.

4.2 TensorRT Export Path & Constraints

The transition from PyTorch to the Jetson-native TensorRT engine is the most critical optimization. I anticipate the following "gotchas":

- Standard PyTorch NMS is often incompatible with TensorRT's fixed-shape requirements. I will utilize the EfficientNMS plugin to move the NMS overhead from the CPU to the GPU.
- Calibration for INT8: To minimize accuracy drop for small-scale UAVs, I will use a representative calibration dataset (500+ images) during the Post-Training Quantization process to ensure the activation tensors for small targets aren't clipped.
- YOLOv11's C3k2 or attention modules may require custom layer implementations if not natively supported by the current TensorRT version on JetPack.

4.3 Jetson Performance Evaluation

To address the performance bottlenecks, I will use NVIDIA Nsight Systems to identify if the bottleneck is in the CUDA kernels (math) or the memory bus (data transfer). Additionally, I would measure the latency between the time the pipeline receives the frame, to the output (and the control signal to the drone), to ensure it complies with the maximal latency.

5 Production Roadmap (Next Steps)

1. Test the pipeline on more videos to draw conclusions about which parts of the pipeline could be removed.
2. Retrain at full-resolution (1280×720) to eliminate SAHI tiling overhead. I will utilize a Teacher-Student distillation framework, using a larger model (e.g. YOLOv11m) to supervise the YOLOv11n student, improving feature extraction while maintaining a lightweight footprint. Collect high quality flying drones frames which match with our drone camera specifications, to bridge the current domain gap in between training and testing data.
3. Flash the NVIDIA Jetson Orin Nano with the JetPack SDK and export the model to a TensorRT FP16/INT8 engine. This allows to verify real-time performance targets of 30–60 FPS through end-to-end latency and throughput benchmarking on the target hardware.
4. The Kalman filter state estimation logic will be refined in two phases:
   - Offline Parameter Identification using a Differentiable Kalman Filter framework. I will utilize a lightweight MLP to learn the optimal static noise covariance matrices (Q, R) by backpropagating the trajectory MSE through the filter's recursive steps, ensuring the estimator is tuned specifically to the aerodynamic profile of the target UAV.
   - Online Adaptive Filtering: Implement an Adaptive Kalman Filter that dynamically adjusts its covariance matrices in real-time to account for the varying maneuverability of the target UAV and transient sensor uncertainty.